

The Imperative Paradigm: Stateful models

The traditional imperative model of computation — to which most of you have been exposed at some stage of your education in a variety of programming languages such as C, C++, Python, Java, Pascal, etc. — has as its computational elements “statements” or “commands” that alter the program *state*.

As expected, these include the following minimal set of commands:

- an identity command — **skip**,
- assignment as the basic operation — **x := e**,
- sequential composition — $c_1; c_2$,
- conditionals — **if e then c₁ else c₂ fi** — and
- repetition — **while e do c₁ od**

The first two constitute elementary commands, and the third, fourth and fifth represent the three different constructions which are expected in any model of computation. The first, namely **skip**, is a left- and right-identity element for the binary operation of sequencing (as one would require in any monoid).

We now present the big-step operational semantic rules for the commands as transformations on Tables. That is, every command takes a given table γ and (if it terminates execution) returns a resulting table γ' . Note that these rules are presented in terms of the structure of the command, and generally, we can use structural induction, except for the **while** construct, since the antecedents of the rule are at least as complex as the consequent. Therefore, a different induction scheme (or co-induction scheme) is used for reasoning about programs with iteration (**while** commands).

- $\gamma \dashv [\text{skip}] \rightarrow \gamma$
- $$\frac{\gamma \vdash e \Rightarrow a}{\gamma \dashv [x := e] \rightarrow \gamma[x \mapsto a]}$$
- $$\frac{\gamma \dashv [c_1] \rightarrow \gamma_1 \quad \gamma_1 \dashv [c_2] \rightarrow \gamma_2}{\gamma \dashv [c_1; c_2] \rightarrow \gamma_2}$$
- $$\frac{\gamma \vdash e \Rightarrow T \quad \gamma \dashv [c_1] \rightarrow \gamma_1}{\gamma \dashv [\text{if } e \text{ then } c_1 \text{ else } c_2 \text{ fi}] \rightarrow \gamma_1}$$
- $$\frac{\gamma \vdash e \Rightarrow F \quad \gamma \dashv [c_2] \rightarrow \gamma_2}{\gamma \dashv [\text{if } e \text{ then } c_1 \text{ else } c_2 \text{ fi}] \rightarrow \gamma_2}$$
- $$\frac{\gamma \vdash e \Rightarrow F}{\gamma \dashv [\text{while } e \text{ do } c_1 \text{ od}] \rightarrow \gamma}$$
- $$\frac{\gamma \vdash e \Rightarrow T \quad \gamma \dashv [c_1] \rightarrow \gamma_1 \quad \gamma_1 \dashv [\text{while } e \text{ do } c_1 \text{ od}] \rightarrow \gamma_2}{\gamma \dashv [\text{while } e \text{ do } c_1 \text{ od}] \rightarrow \gamma_2}$$

Hoare Logic

CAR (Tony) Hoare in 1969 wrote one of the most influential papers in the science of computing called *An Axiomatic Basis for Computing*, in which he provided a very strong link between logic and programming. This was following up on work by Robert (Bob) Floyd, who provided annotated flowcharts with logical properties to be able to reason about the correctness of programs, and Edgser W Dijkstra (EWD) in providing an formal basis for reasoning about programs and their correctness with notions of weakest preconditions and strongest postconditions, program invariants, and with connections to logic.

Hoare noted that a state is a valuation which associates answers (or values) to variables, and the truth value of a (first-order logic) predicate is dependent on exactly such a valuation. Thus, we can evaluate a first-order logic formula in a state, and conversely, we can characterise a set of states using a first-order logic formula:

Let $\gamma \models \varphi$ be the notation for stating that formula φ is true in (or “is satisfied by”) state γ . The set of states satisfying formula φ is given by $\{\gamma \mid \gamma \models \varphi\}$. Now instead of reasoning about individual execution behaviours (traces) of a program c starting with a state γ , we can symbolically reason about the set of states in which the program c terminates when starting with a state satisfying the predicate φ . Or even more usefully, to determine the set of states in which we may start in order to ensure that the program will terminate in a state satisfying a formula ψ . (We want our outputs to satisfy a given property, e.g., being a sorted array. Or variable i being equal to $n!$)

Hoare also used a very clever observation that one could treat program expressions (including variables) as expressions in the first-order logic formulae (respectively variables in the logic). This double role played by program variable x being synonymous with the variable x in a logical formula is crucial for making the axiomatic framework operate as a *symbolic execution* of the program (that is reasoning about program execution without actually executing the program). When this mapping is disrupted (e.g., when the named storage in the program is not expressed as an expression in the logic, or we have aliasing or pointers, the axioms do not work smoothly, and we need to place all manner of other conditions on the rules).

Hoare presented the notion of what is now called a Hoare triple $\{\varphi\} c \{\psi\}$, which is intended to mean that if one starts executing the command c in a state that satisfies the formula φ then, if executing c terminates, it is guaranteed to do so in a state that satisfies the formula ψ . The triple can also be read *backwards*: if one wishes to reach a state satisfying the formula ψ by executing the command c then it suffices to start in a state satisfying the formula φ .

[Note: The original notation was different — it was $\varphi \{c\} \psi$ — but placing (set-theoretic) braces around the formulas instead of the the commands conveys better the intuition that we are talking of sets of states that satisfy the formula. The

backwards reading of the triple is the more interesting one in terms of proving program correctness.]

For the small imperative language we have presented, Hoare's axioms are as follows:

- [Skip] $\{\varphi\} \text{ skip } \{\varphi\}$
- [Assignment] $\{\varphi[e/x]\} x := e \{\varphi\}$
- [Sequential]
$$\frac{\{\varphi\} c_1 \{\varphi_1\} \quad \{\varphi_1\} c_2 \{\varphi_2\}}{\{\varphi\} c_1; c_2 \{\varphi_2\}}$$
- [Conditional T]
$$\frac{\{\varphi \wedge e\} c_1 \{\varphi_1\}}{\{\varphi\} \text{ if } e \text{ then } c_1 \text{ else } c_2 \text{ fi } \{\varphi_1\}}$$
- [Conditional F]
$$\frac{\{\varphi \wedge \neg e\} c_2 \{\varphi_2\}}{\{\varphi\} \text{ if } e \text{ then } c_1 \text{ else } c_2 \text{ fi } \{\varphi_2\}}$$
- [While]
$$\frac{\{\varphi \wedge e\} c_1 \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } c_1 \text{ od} \rightarrow \{\varphi \wedge \neg e\}}$$
- [Consequence]
$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} c \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} c \{\psi'\}}$$

Explanation of the rules

The [Skip] rule says that if we execute the command `skip` starting with a state γ satisfying φ we will reach a state (the same γ !) that satisfies φ .

The [Assignment] rule is a subtle rule that requires some imagination: If you wish to reach a state that satisfies formula φ , by executing an assignment command $x := e$ then one can do so by starting from a state that satisfies $\varphi[e/x]$, i.e., φ with the rhs expression esubstituted for the assigned variable x .

The [Sequential] rule chains the conditions for commands c_1 and c_2 : if starting from *any* state that satisfies φ and executing command c_1 results in a state satisfying φ_1 , and starting from *any* state satisfying φ_1 executing command c_2 results in a state satisfying φ_2 , then executing the composite command $c_1; c_2$ starting from any state satisfying φ will result (on termination) in a state satisfying φ_2 .

The two [Conditional $_$] rules are complementary: if starting from *any* state that satisfies φ in which e is also satisfied, executing command c_1 results in a state satisfying φ_1 , then starting from a state that satisfying φ , executing the command `if  $e$  then  $c_1$  else  $c_2$  fi` will result in a state satisfying φ_1 . Complementarily, if starting from *any* state that satisfies φ in which e is not satisfied (i.e., $\neg e$ holds), executing command c_2 results in a state satisfying φ_2 , then starting from a state that satisfying φ , executing the command `if  $e$  then  $c_1$  else  $c_2$  fi` will result in a state satisfying φ_2 .

The [While] rule involves the notion of a “loop invariant”. If executing the body of the loop, command c_1 starting from a state satisfying φ in which e is also satisfied results in a state that continues to satisfy φ — it holds at the top and the end of the loop body — then executing the command `while  $e$  do  $c_1$  od` starting from a state in which the formula φ holds will, on termination of the loop, result in a state where $\varphi \wedge \neg e$ holds.

Note that the first 6 rules are based on the abstract syntax.

The only rule that is not “syntax-directed” is the last rule. This rule involves the notion of *logical consequence*. A formula φ_2 is a logical consequence of another formula φ_1 , written $\varphi_1 \Rightarrow \varphi_2$, if for every state γ that satisfies φ_1 , it is also the case that state γ satisfies φ_2 . The [Consequence] rule is very important for piecing together proofs of correctness: it states that if the Hoare triple $\{\varphi\} c \{\psi\}$ has been proven, and if $\varphi' \Rightarrow \varphi$ and $\psi \Rightarrow \psi'$, then the Hoare triple $\{\varphi'\} c \{\psi'\}$ holds. That is, we can “strengthen” the precondition from φ to φ' , and “weaken” the post-condition from ψ to ψ' . (Strengthening a condition means fewer states can satisfy it; F is the strongest condition since *no* state satisfies it. Weakening a condition allows more states to satisfy it; T is the weakest condition since *every* state satisfies it.)

An Example

Let us see how the Hoare rules work in a simple program to compute factorial of a natural number n .

```

    -  $0 \leq n$ 
    -  $1 = 0! \wedge (0 \leq n)$  [Consequence]
i := 0;
    -  $1 = i! \wedge (i \leq n)$  [Assignment]
x := 1;
    -  $x = i! \wedge (i \leq n)$  [Assignment]
while i < n do
    -  $x = i! \wedge (i \leq n) \wedge (i < n)$  [While antecedent]
    -  $x = i! \wedge (i < n)$  [Consequence]
    -  $x * (i + 1) = (i + 1)! \wedge (i + 1 \leq n)$  [Assignment]
    i := i + 1;
    -  $x * i = i! \wedge (i \leq n)$  [Assignment]
    x := x * i
    -  $x = i! \wedge (i \leq n)$  [Assignment]
od
    -  $x = i! \wedge (i \leq n) \wedge \neg(i < n)$  [While]
    -  $x = n!$  [Consequence]
```

Theorem (*Soundness of Hoare Logic*) For any command c , for all formulas φ and ψ , if $\varphi \{c\} \psi$ is provable from the axioms and inference rules of Hoare logic, and for any state γ if $\gamma \models \varphi$, then if execution of c terminates in some state γ' , then $\gamma' \models \psi$.

Proof is by structural induction on the structure of the proof tree of with root (conclusion) $\varphi \{c\} \psi$. Analysis is based on the last rule used (nearest the root of the proof tree).

Exercise (**) Prove the Soundness of Hoare Logic for this core imperative language. Show that each rule is operationally sound, and so by induction, any proof tree is operationally sound.